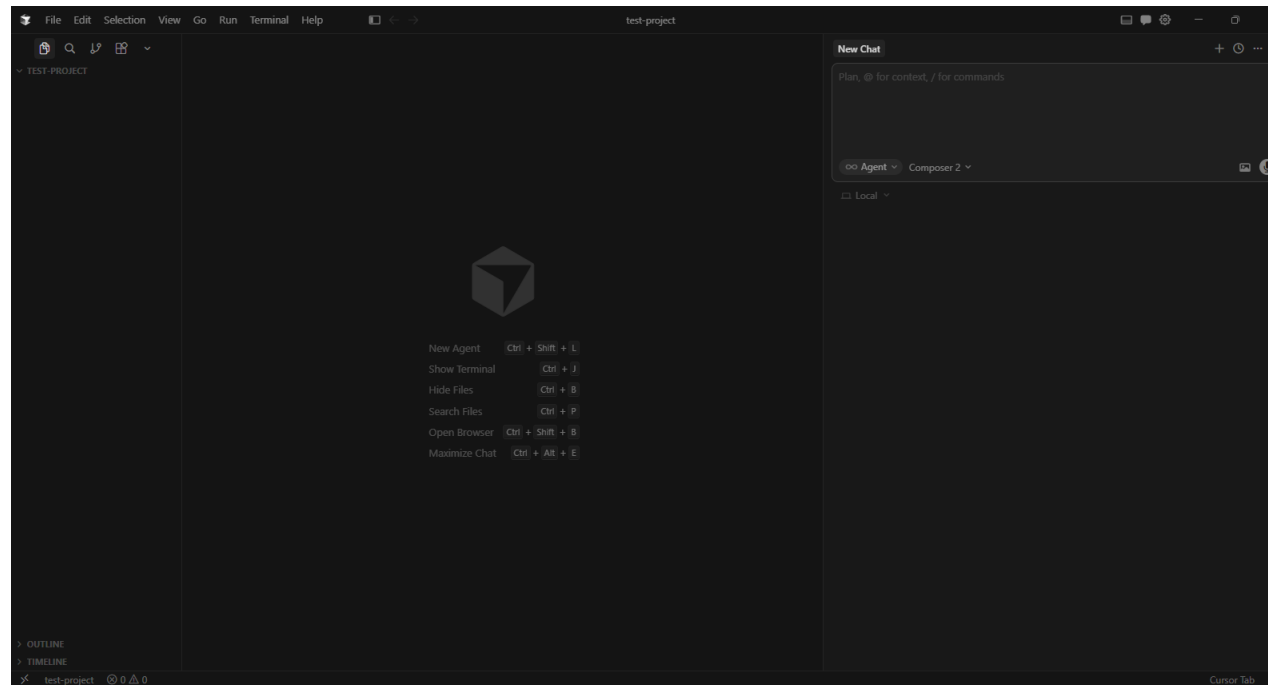


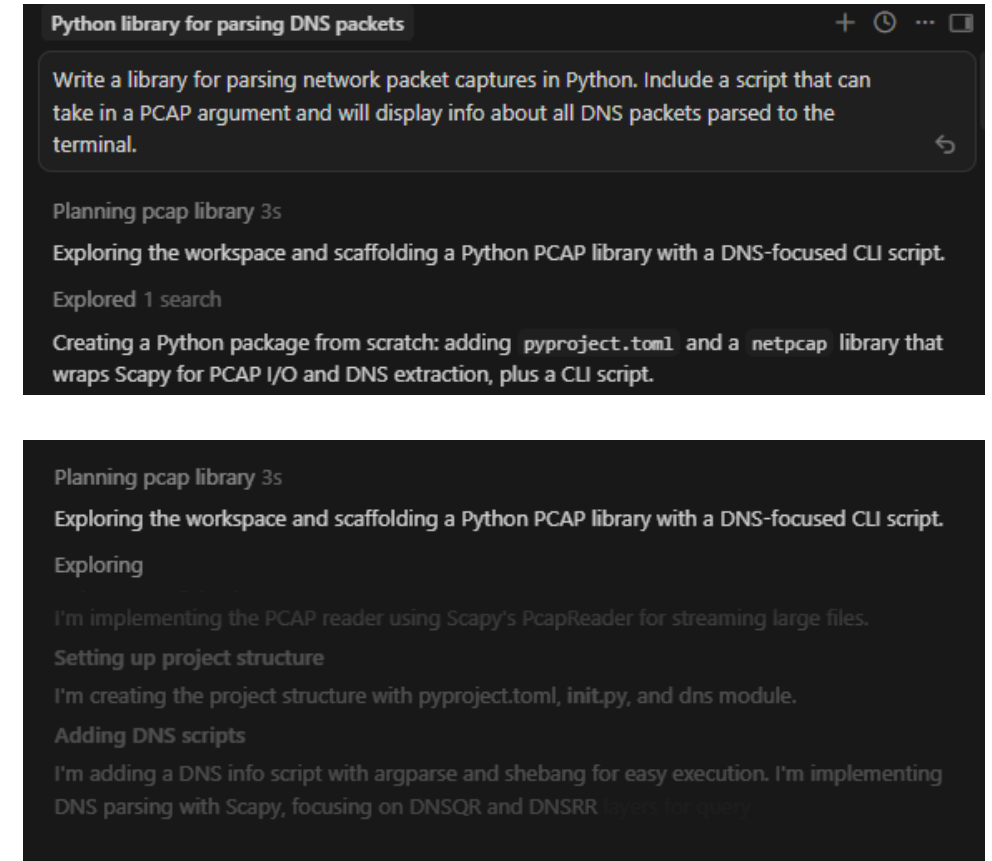
First Exercise: Vibe Coding

- **Step 1:** Open blank project
 - Make a “test-project” folder in your home directory to hold your project
 - Run cursor (cursor.dev?) (already installed? Set API key?)
 - File → Open Folder → “test-project”

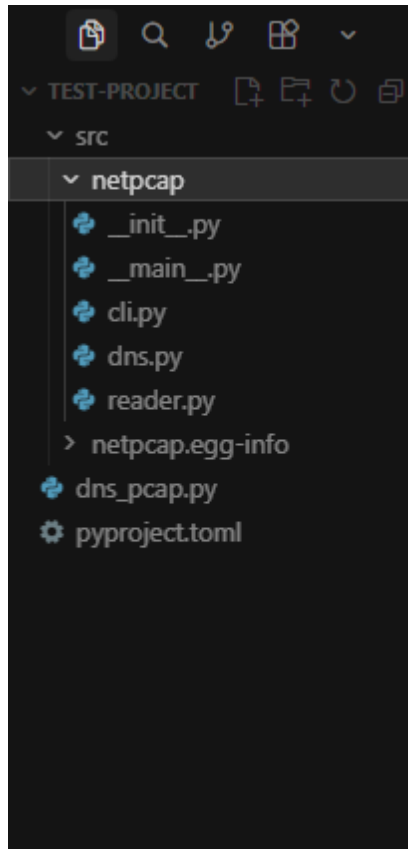


First Exercise: Vibe Coding

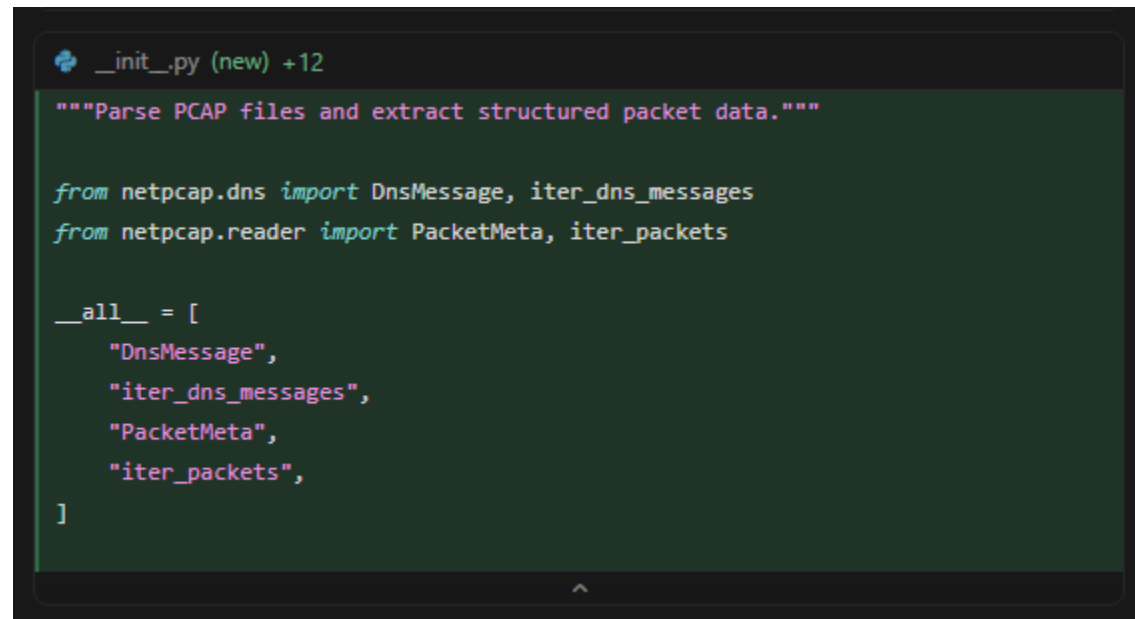
- **Step 2:** Ask the AI to create your initial project
 - In the chat window on the right:
 - *“Write a library for parsing network packet captures in Python. Include a script that can take in a PCAP argument, and will display info about all DNS packets parsed to the terminal.”*
 - In the chat window, you can see the AI working through the problem step-by-step. You can click on one of the summaries in the chat window to see what the AI is “thinking.” These are the output tokens that the AI is producing on its way to producing code.



First Exercise: Vibe Coding

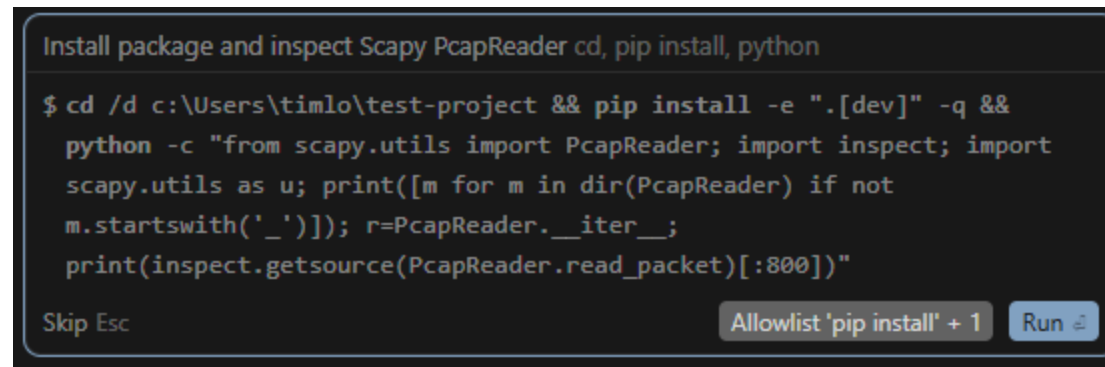


- On the left, you can see the project's files being filled in to whatever structure the AI decided on.
- You can see lines of code being added in the chat window too.



First Exercise: Vibe Coding

- The AI needs **permission** to do things: perform web searches, write files, etc.
 - Discussion on allowlists, safety, etc.
 - How could this be abused? What sort of policy should you use to allow some things but not others? Best practices?



```
Install package and inspect Scapy PcapReader cd, pip install, python

$ cd /d c:\Users\timlo\test-project && pip install -e ".[dev]" -q &&
python -c "from scapy.utils import PcapReader; import inspect; import
scapy.utils as u; print([m for m in dir(PcapReader) if not
m.startswith('_')]); r=PcapReader.__iter__;
print(inspect.getsource(PcapReader.read_packet)[:800])"
```

Skip Esc Allowlist 'pip install' + 1 Run

First Exercise: Vibe Coding

- Wait for the AI to finish. It will look something like this when done.

```
reader.py x
src > netpcap > reader.py
1 """Stream packets from a PCAP file with basic metadata."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 from pathlib import Path
7 from typing import Iterator, Optional
8
9 from scapy.layers.inet import IP
10 from scapy.layers.inet6 import IPv6
11 from scapy.packet import Packet
12 from scapy.utils import PcapReader
13
14
15 @dataclass(frozen=True)
16 class PacketMeta:
17     """One packet from a capture with timing and optional IP endpoints."""
18
19     index: int
20     timestamp: float
21     raw: Packet
22     src_ip: Optional[str]
23     dst_ip: Optional[str]
24
25
26 def _ip_endpoints(pkt: Packet) -> tuple[Optional[str], Optional[str]]:
27     if IP in pkt:
28         ip = pkt[IP]
29         return str(ip.src), str(ip.dst)
30     if IPv6 in pkt:
31         ip6 = pkt[IPv6]
32         return str(ip6.src), str(ip6.dst)
33     return None, None
34
35
36 def iter_packets(pcap_path: str | Path) -> Iterator[PacketMeta]:
37     """
38     Yield each packet from *pcap_path* without loading the whole file into memory.
```

Usage

```
pip install -e .
dns-pcap your_capture.pcap
```

or, from the project root without install:

```
python dns_pcap.py your_capture.pcap
```

Library example:

```
from netpcap import iter_dns_messages

for msg in iter_dns_messages("capture.pcap"):
    print(msg.packet_index, msg.is_response, msg.questions)
```

Scapy handles PCAP/PCAPNG dissection; the library focuses on a small streaming API and structured DNS views. If `dns-pcap` is not on your PATH after install, use `python dns_pcap.py` or `python -m netpcap` as above.

Worked for 19m

> 7 Files

Undo All

Keep All

Review

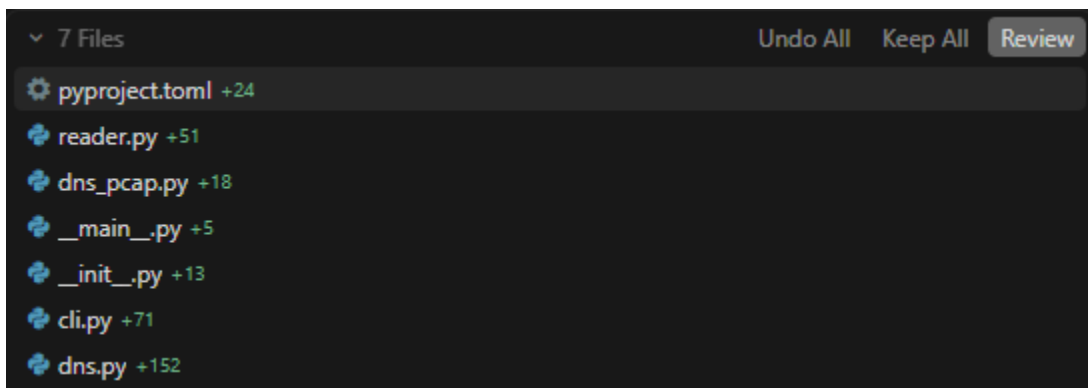
plan, @ for context, / for commands

Agent

Composer 2

First Exercise: Vibe Coding

- **Step 3:** Review what happened.
 - The AI will output a summary into the chat window at the end of what it did.
 - You can review the full set of changes (in this case, whole files) the AI wrote by expanding the bottom right. You can click through them to review and accept/reject each change.
 - Does it look good?



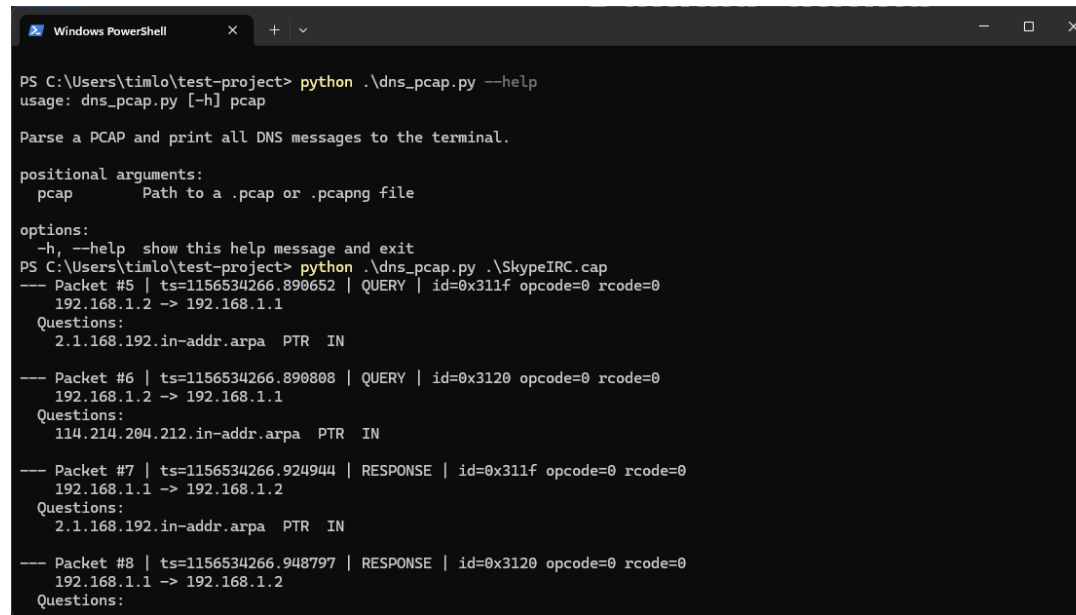
Here's what was added and how to use it.

Layout

- `pyproject.toml` — Package `netpcap`, dependency `Scapy`, console entry point `dns-pcap`.
- `src/netpcap/reader.py` — Streams PCAPs without loading the whole file: `iter_packets()` yields `PacketMeta` (1-based index, timestamp, raw Scapy packet, IPv4/IPv6 src/dst when present).
- `src/netpcap/dns.py` — `parse_dns_from_packet()`, `iter_dns_messages()`, `dns_from_packet()`, plus `DnsMessage` / `DnsQuestion` / `DnsAnswer` dataclasses (questions + answers/authority/additional, with IANA-style type/class names).
- `src/netpcap/cli.py` — Argument parser and terminal formatting.
- `dns_pcap.py` (repo root) — Adds `src` to `sys.path` so you can run `python dns_pcap.py capture.pcap` without installing.
- `src/netpcap/__main__.py` — Supports `python -m netpcap` after install.

First Exercise: Vibe Coding

- **Step 4:** Run it. Does it seem to work?
 - Some AI runs may have automatically produced their own tests. Should we trust those?
 - You can grab a test file with DNS traffic here:
<http://wiki.wireshark.org/samplecaptures>



```
Windows PowerShell
PS C:\Users\timlo\test-project> python .\dns_pcap.py --help
usage: dns_pcap.py [-h] pcap

Parse a PCAP and print all DNS messages to the terminal.

positional arguments:
  pcap                Path to a .pcap or .pcapng file

options:
  -h, --help          show this help message and exit
PS C:\Users\timlo\test-project> python .\dns_pcap.py .\SkypeIRC.cap
--- Packet #5 | ts=1156534266.890652 | QUERY | id=0x311f opcode=0 rcode=0
192.168.1.2 -> 192.168.1.1
Questions:
2.1.168.192.in-addr.arpa PTR IN

--- Packet #6 | ts=1156534266.890808 | QUERY | id=0x3120 opcode=0 rcode=0
192.168.1.2 -> 192.168.1.1
Questions:
114.214.204.212.in-addr.arpa PTR IN

--- Packet #7 | ts=1156534266.924944 | RESPONSE | id=0x311f opcode=0 rcode=0
192.168.1.1 -> 192.168.1.2
Questions:
2.1.168.192.in-addr.arpa PTR IN

--- Packet #8 | ts=1156534266.948797 | RESPONSE | id=0x3120 opcode=0 rcode=0
192.168.1.1 -> 192.168.1.2
Questions:
```

First Exercise: Vibe Coding

- **Step 5 (optional):** Debug problems
 - If needed, ask the AI to fix any problems you find (output wrong, crash, etc.) You can often paste the error directly into the chat to get it started.
 - The first time I ran this tutorial, it failed the first test and needed fixes. The second time it worked right away. Shrug

First Exercise: Vibe Coding

- **Step 6:** Add cool visualization feature!
 - *“Add a new option to the script that can produce a visualization from the provided capture.”*

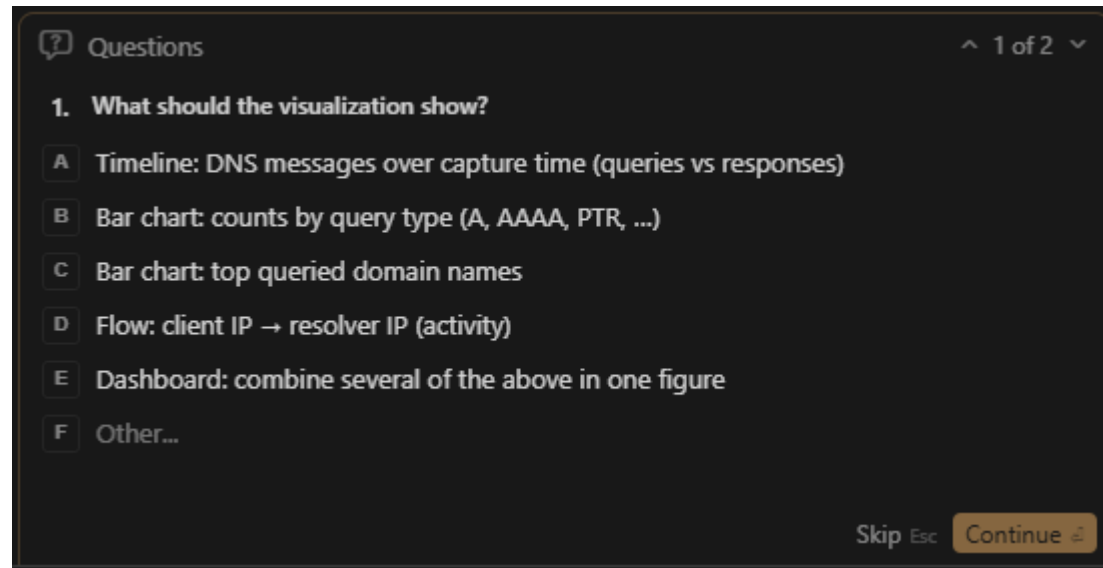
Add a new option to the script that can produce a visualization from the provided capture.

Thought briefly

Reviewing the CLI and DNS structures, then adding a visualization option (likely matplotlib) that generates a plot from the capture.

First Exercise: Vibe Coding

- Sometimes the AI might ask you a question.
- You can just click the multiple choice answer you like best, or give it your own custom answer.



A screenshot of a dark-themed dialog box titled "Questions" with a question mark icon. In the top right corner, it says "1 of 2" with up and down arrows. The main content is a list of options for a question: "1. What should the visualization show?". The options are labeled A through F, each with a small square icon to its left. At the bottom right, there are two buttons: "Skip" with "Esc" next to it, and "Continue" with a right arrow icon.

Questions ^ 1 of 2 v

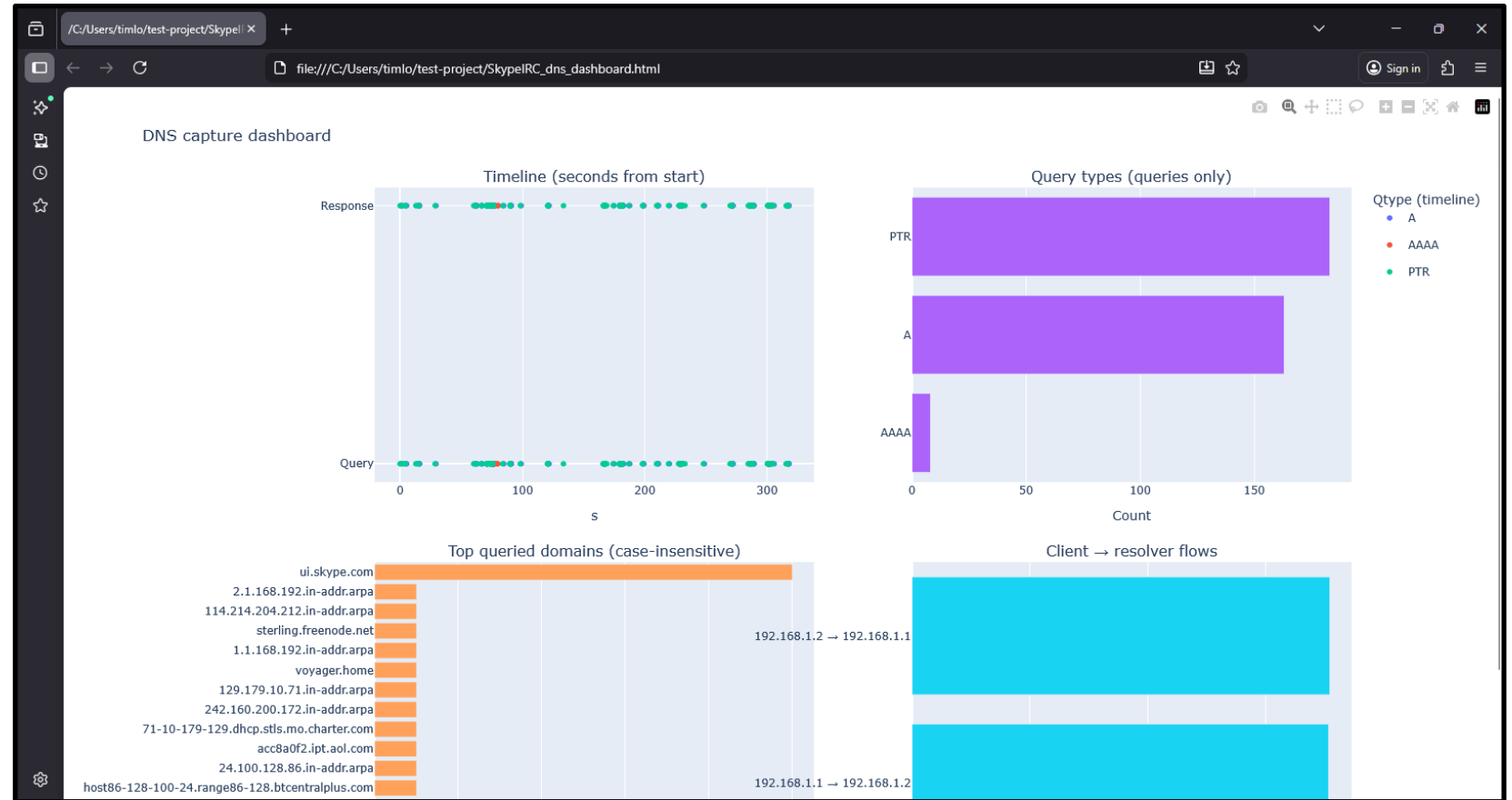
1. What should the visualization show?

- ☐ A Timeline: DNS messages over capture time (queries vs responses)
- ☐ B Bar chart: counts by query type (A, AAAA, PTR, ...)
- ☐ C Bar chart: top queried domain names
- ☐ D Flow: client IP → resolver IP (activity)
- ☐ E Dashboard: combine several of the above in one figure
- ☐ F Other...

Skip Esc Continue →

First Exercise: Vibe Coding

- Then try out the visualization:



First Exercise: Vibe Coding

- This concludes the Vibe Coding exercise. Feel free to play around with Cursor for a bit.
- Bonus content if done early